

- 1) b) Data and a pointer to next node
- 2) c) NULL (NULL = null & char built)
- 3) b) Self-referential structure
- 4) b) head == NULL
- 5) b) newnode (new = malloc) & data
- 6) b) temp = head; head = head->next; Free(temp);
- 7) c) Traversing from tail to head
- 8) b) n
- 9) c) Copying the data of P $\rightarrow$ next into P, then deleting P $\rightarrow$ next
- 10) b) while(p != NULL)
- 11) b) p->next == NULL
- 12) c) Two pointers advancing at
- 13) c) Three
- 14) b) Exactly one node
- 15) b) p = (struct node*) malloc(sizeof(struct node));
- 16) b) Avoid special-case handling for insertion and deletion at the head
- 17) c) First Node
- 18) b) It does not support random (indexed) access
- 19) b) Loss of access to all nodes, causing a memory leak
- 20) b) A Slow pointer and a Fast pointer
- 21) c) The Last Node
- 22) c) Size must be declared in advance
- 23) struct node {
int data;
struct node *next;
- 24) 1 P=10 10+20=30
2 P=20 20+30=50
3 P=30 30+40=70
4 P=40 (Fails condition)
Output 30 50 70

26.

```
struct node * reverseList (struct node * head)
```

```
struct node * prev = NULL;
```

```
struct node * current = head;
```

```
struct node * next = NULL;
```

```
while (current != NULL) {
```

```
next = current->next;
```

```
current->next = prev;
```

```
prev = current;
```

```
}
```

```
return prev;
```